DESARROLLO FULL STACK · UNIDAD 2

Bases de datos relacionales y NoSQL

Back-end como Servicio (BaaS) y construcción de una base de datos



90 minutos | Teórico-práctica | PostgreSQL · MongoDB · Supabase · Firebase

Escuela de Ingeniería de Sistemas · Docente: [Nombre del docente]

RUTA DE LA SESIÓN · MODELO IUTPC

Cinco momentos en 90 minutos

INICIO

10’

Pregunta reto y logro de la sesión

UTILIDAD

5’

Dónde encaja la base de datos en el stack

TRANSFORMACIÓN

45’

SQL, NoSQL, BaaS y construcción de una BD

PRÁCTICA

25’

Laboratorio: TaskFlow con Supabase y React

CIERRE

5’

Metacognición y entregable

INICIO · PREGUNTA RETO

Una app de delivery guarda pedidos, pagos, la ubicación del repartidor en vivo y un chat. ¿Dónde vive cada dato?

- **Pedidos y pagos**
- **Ubicación en vivo**
- **Chat cliente–repartidor**
- **Reseñas e historial**

Discute en parejas (2 min): ¿basta con una sola base de datos o conviene usar varias? ¿Por qué?

INICIO · LOGRO DE LA SESIÓN

Diseñas, construyes y consumes una base de datos real

Al finalizar la sesión, el estudiante **diseña y construye una base de datos relacional en un BaaS (Supabase)** y la **consume desde una aplicación React**, justificando cuándo usar un modelo relacional o uno NoSQL.

Modela

Diagrama entidad-relación normalizado hasta la 3FN.

Construye

Tablas, claves, restricciones y políticas RLS en Supabase.

Consume

Lista, crea y actualiza datos desde React con supabase-js.

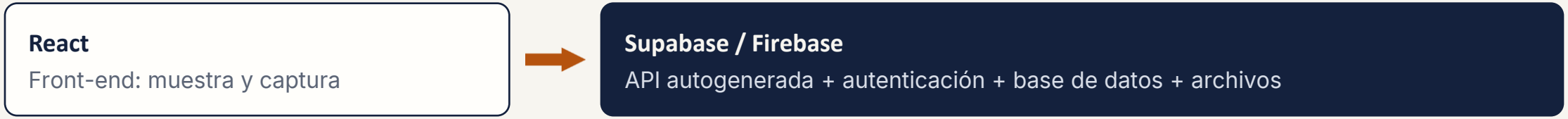
UTILIDAD · ¿PARA QUÉ ME SIRVE?

La base de datos es la memoria de tu aplicación

Con back-end propio (lo que viste en la Unidad 1)



Con Back-end como Servicio (lo que veremos hoy)



Sin persistencia, al recargar la página todo se pierde: **el front-end muestra, la base de datos recuerda.**

TRANSFORMACIÓN · FUNDAMENTOS

Base de datos, SGBD y modelo de datos

Base de datos

Colección organizada y persistente de datos relacionados. Ejemplo: los clientes y pedidos de una tienda.

SGBD

Software que almacena, consulta, protege y respalda los datos. Ejemplos: PostgreSQL, MySQL, MongoDB.

Modelo de datos

La forma de organizar los datos: tablas, documentos, pares clave-valor o grafos.

Elegir el modelo es una **decisión de arquitectura**: condiciona cómo consultas, escalas y aseguras tus datos.

TRANSFORMACIÓN · MODELO RELACIONAL

Datos en tablas que se relacionan por claves

Tabla · una entidad (clientes)

Fila · un registro concreto

Columna · un atributo con tipo

Clave primaria (PK) · identifica cada fila

Clave foránea (FK) · apunta a la PK de otra tabla

clientes

id	nombre	ciudad
1	Ana Quispe	Lima
2	Luis Rojas	Arequipa
3	María Huamán	Lima

pedidos

id	cliente_id	total
1	1	120.50
2	1	80.00
3	2	45.90

pedidos.cliente_id → clientes.id : Ana tiene 2 pedidos, Luis 1 y María ninguno.

EJEMPLO 1 · SQL (DDL)

Definir la estructura: CREATE TABLE

```sql
-- Ejemplo 1: tienda en PostgreSQL
CREATE TABLE clientes (
  id      SERIAL PRIMARY KEY,
  nombre  VARCHAR(80)  NOT NULL,
  email   VARCHAR(120) UNIQUE NOT NULL,
  ciudad  VARCHAR(50)
);

CREATE TABLE pedidos (
  id        SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES clientes(id),
  total     NUMERIC(10,2) CHECK (total > 0),
  fecha     DATE DEFAULT CURRENT_DATE
);
```

▶ Resultado en psql

```
CREATE TABLE
CREATE TABLE   -- 2 tablas creadas
```

PRIMARY KEY

Identifica cada fila; SERIAL genera el número automáticamente.

REFERENCES

Clave foránea: un pedido solo puede apuntar a un cliente que exista.

CHECK · UNIQUE · NOT NULL

Reglas que el SGBD hace cumplir por ti.

EJEMPLO 1 · SQL (DML)

Insertar y consultar con JOIN

```sql
INSERT INTO clientes (nombre, email, ciudad) VALUES
 ('Ana Quispe',   'ana@mail.pe',   'Lima'),
 ('Luis Rojas',   'luis@mail.pe',  'Arequipa'),
 ('María Huamán', 'maria@mail.pe', 'Lima');
INSERT INTO pedidos (cliente_id, total, fecha) VALUES
 (1, 120.50, '2026-10-01'), (1, 80.00, '2026-10-03'),
 (2, 45.90, '2026-10-02');

SELECT c.nombre, COUNT(p.id) AS pedidos,
       COALESCE(SUM(p.total), 0) AS gastado
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.nombre
ORDER BY gastado DESC;
```

▶ **Resultado**

nombre	pedidos	gastado
Ana Quispe	2	200.50
Luis Rojas	1	45.90
María Huamán	0	0.00

(3 filas)

¿Por qué LEFT JOIN?

Conserva a María aunque no tenga pedidos. Con INNER JOIN desaparecería del reporte.

TRANSFORMACIÓN · CLAVES COMPUESTAS

Clave compuesta: cuando una columna no basta

Una **clave primaria compuesta** (o combinada) está formada por dos o más columnas: ninguna identifica la fila por sí sola, pero su **combinación** es única.

matriculas · PK = (alumno_id, curso_id, periodo)

alumno_id	curso_id	periodo	nota
1	10	2026-2	16
1	11	2026-2	14
2	10	2026-2	18
1	10	2027-1	—

El alumno 1 se repite, el curso 10 se repite, el periodo 2026-2 se repite… pero la terna nunca.

Cuándo usarla

Tablas intermedias N:M (alumno–curso, tarea–etiqueta) e identificadores naturales de varias partes (serie + número de factura).

Alternativa común

id SERIAL como PK + UNIQUE (alumno_id, curso_id, periodo). Útil con ORM y rutas REST como /matriculas/57.

EJEMPLO 1B · CLAVE COMPUESTA

Definirla y comprobar que protege los datos

```sql
-- Ejemplo 1b: matrícula con clave primaria compuesta
CREATE TABLE matriculas (
  alumno_id INT REFERENCES alumnos(id),
  curso_id  INT REFERENCES cursos(id),
  periodo   CHAR(6),
  nota      NUMERIC(4,2) CHECK (nota BETWEEN 0 AND 20),
  PRIMARY KEY (alumno_id, curso_id, periodo)
);

INSERT INTO matriculas VALUES (1, 10, '2026-2', NULL); -- ok
INSERT INTO matriculas VALUES (1, 11, '2026-2', NULL); -- ok
INSERT INTO matriculas VALUES (1, 10, '2027-1', NULL); -- ok
INSERT INTO matriculas VALUES (1, 10, '2026-2', NULL); -- ¿?
```

▶ Resultado del 4.º INSERT

```
ERROR: duplicate key value violates
  unique constraint "matriculas_pkey"
DETAIL: Key (alumno_id, curso_id,
  periodo)=(1, 10, 2026-2) already
  exists.
```

▶ SELECT * FROM matriculas

alumno_id	curso_id	periodo
1	10	2026-2
1	11	2026-2
1	10	2027-1

EJEMPLO 1B · CLAVE COMPUESTA

Usarla: clave foránea compuesta y consultas

```
CREATE TABLE asistencias (
  alumno_id INT,
  curso_id  INT,
  periodo   CHAR(6),
  fecha     DATE,
  presente  BOOLEAN DEFAULT true,
  PRIMARY KEY (alumno_id, curso_id, periodo, fecha),
  FOREIGN KEY (alumno_id, curso_id, periodo)
    REFERENCES matriculas (alumno_id, curso_id, periodo)
);

-- el alumno 2 NO está matriculado en el curso 10 en 2026-2
INSERT INTO asistencias VALUES (2, 10, '2026-2', '2026-10-06');
```

▶ Resultado

```
ERROR: insert or update on table
  "asistencias" violates foreign key
  constraint "asistencias_..._fkey"
DETAIL: Key (alumno_id, curso_id,
  periodo)=(2, 10, 2026-2) is not
  present in table "matriculas".
```

Referenciarla

La FK debe tener las mismas columnas, en el mismo orden. En los JOIN se comparan todas: ON (a.alumno_id, a.curso_id, a.periodo) = (m.alumno_id, m.curso_id, m.periodo).

El orden importa: el índice de la PK sirve para filtrar por alumno_id, o por alumno_id + curso_id; un filtro solo por curso_id no lo aprovecha bien.

CLAVES COMPUESTAS · EJEMPLOS

Seis casos reales de clave combinada

detalle_pedido

PK (pedido_id, producto_id)

Evita: el mismo producto en dos líneas del mismo pedido.

seguidores

PK (seguidor_id, seguido_id)

Evita: seguir dos veces a la misma cuenta.

stock

PK (almacen_id, producto_id)

Evita: dos saldos distintos del mismo producto en un almacén.

comprobantes

PK (serie, numero)

Evita: emitir dos veces la factura F001-00000123.

horarios

PK (aula_id, dia, hora_inicio)

Evita: dos clases en la misma aula a la misma hora.

tarea_etiqueta

PK (tarea_id, etiqueta_id)

Evita: poner dos veces la misma etiqueta a una tarea.

En todos los casos, la **regla de negocio** («no puede repetirse esta combinación») es la que define la clave.

COMPUESTA VS. ID · DOS DISEÑOS

La misma tabla, dos formas de identificar la fila

```sql
-- Diseño A: clave compuesta
CREATE TABLE detalle_pedido (
  pedido_id   INT REFERENCES pedidos,
  producto_id INT REFERENCES productos,
  cantidad    INT CHECK (cantidad > 0),
  precio      NUMERIC(10,2),
  PRIMARY KEY (pedido_id, producto_id)
);
```

Qué obtienes

La regla de negocio **es** la identidad. NOT NULL automático. Un solo índice.

```sql
-- Diseño B: clave sustituta + UNIQUE
CREATE TABLE detalle_pedido (
  id          SERIAL PRIMARY KEY,
  pedido_id   INT NOT NULL REFERENCES pedidos,
  producto_id INT NOT NULL REFERENCES productos,
  cantidad    INT CHECK (cantidad > 0),
  precio      NUMERIC(10,2),
  UNIQUE (pedido_id, producto_id)
);
```

Qué obtienes

Identidad simple (**id**); la regla vive en **UNIQUE**. NOT NULL lo declaras tú. Dos índices.

Los dos diseños impiden duplicados. El problema aparece con un tercero: **id sin UNIQUE**.

COMPUESTA VS. ID · SIN CLAVE COMBINADA

El error silencioso: id sin UNIQUE

```
-- Diseño C: id SIN restricción de unicidad
CREATE TABLE detalle_pedido (
  id          SERIAL PRIMARY KEY,
  pedido_id   INT REFERENCES pedidos,
  producto_id INT REFERENCES productos,
  cantidad    INT,
  precio      NUMERIC(10,2)
);
-- el cliente hace doble clic en «Agregar al carrito»
INSERT INTO detalle_pedido
  (pedido_id, producto_id, cantidad, precio)
VALUES (1, 7, 2, 35.00);
INSERT INTO detalle_pedido
  (pedido_id, producto_id, cantidad, precio)
VALUES (1, 7, 2, 35.00);
```

▶ Resultado

```
INSERT 0 1
INSERT 0 1   -- ¡ambos aceptados!
```

▶ SELECT * FROM detalle_pedido

id	pedido	producto	cant.
1	1	7	2
2	1	7	2

Impacto

El pedido suma S/ 140 en lugar de S/ 70, el stock se descuenta dos veces y el reporte de ventas se infla.

COMPUESTA VS. ID · IMPACTO EN TABLAS HIJAS

Cuando otra tabla necesita referenciarla

```sql
-- Hija con diseño A
CREATE TABLE devoluciones (
  id         SERIAL PRIMARY KEY,
  pedido_id  INT,
  producto_id INT,
  motivo     TEXT,
  FOREIGN KEY (pedido_id, producto_id)
    REFERENCES detalle_pedido
);
SELECT d.motivo, dp.cantidad
FROM devoluciones d JOIN detalle_pedido dp
  ON  dp.pedido_id  = d.pedido_id
  AND dp.producto_id = d.producto_id;
```

```sql
-- Hija con diseño B
CREATE TABLE devoluciones (
  id               SERIAL PRIMARY KEY,
  detalle_pedido_id INT REFERENCES detalle_pedido,
  motivo           TEXT
);
SELECT d.motivo, dp.cantidad
FROM devoluciones d JOIN detalle_pedido dp
  ON dp.id = d.detalle_pedido_id;
```

Diseño A

FK de varias columnas: JOIN más largo y riesgo de olvidar una. A favor: conoce pedido_id sin JOIN.

Diseño B

Una columna: JOIN simple, URL /detalles/57 y mapeo directo en JPA o Prisma.

COMPUESTA VS. ID · ¿CUÁL ELIJO?

Comparación de los tres diseños

Criterio	A · Clave compuesta	B · id + UNIQUE	C · id sin UNIQUE
Impide duplicados	Sí	Sí	No
FK en tablas hijas	2 o más columnas	1 columna	1 columna
JOIN	Por todas las columnas	Por id	Por id
API REST / ORM	/pedidos/1/productos/7	/detalles/57	/detalles/57
Cambiar un valor de la clave	Se propaga a las hijas	Solo se edita la fila	Solo se edita la fila
Índices	1 (la PK)	2 (PK y UNIQUE)	1, sin proteger la regla
Ideal para	Tablas puente sin hijas	Entidades con hijas o expuestas en API	Datos que sí pueden repetirse

Regla práctica: tabla puente sin hijas → compuesta. Si la referencian otras tablas o una API → id + UNIQUE. Nunca id solo si la combinación no debe repetirse.

TRANSFORMACIÓN · TRANSACCIONES

ACID: la garantía de las bases relacionales

A

Atomicidad

Todo o nada: la transferencia se completa entera o no ocurre.

C

Consistencia

Solo se pasa de un estado válido a otro: se respetan PK, FK y CHECK.

I

Aislamiento

Las transacciones simultáneas no ven cambios a medio hacer.

D

Durabilidad

Tras el COMMIT, el dato sobrevive a un corte de luz.

```
BEGIN;
  UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
  UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;   -- si algo falla antes: ROLLBACK y nada cambia
```

TRANSFORMACIÓN · NOSQL

NoSQL («Not only SQL»): cuatro familias

Esquemas flexibles y escalado horizontal para datos que cambian de forma o crecen muy rápido.

Documentos

Motores: MongoDB, Firestore

Uso: perfiles, catálogos, contenido

Clave-valor

Motores: Redis, DynamoDB

Uso: caché, sesiones, carritos

Columnas anchas

Motores: Cassandra, HBase

Uso: IoT, logs, series de tiempo

Grafos

Motores: Neo4j, Amazon Neptune

Uso: redes sociales, recomendaciones

EJEMPLO 2 · MONGODB

Un documento guarda al cliente con sus pedidos

```
// Ejemplo 2: los mismos clientes en MongoDB
db.clientes.insertOne({
  nombre: "Ana Quispe",
  email: "ana@mail.pe",
  ciudad: "Lima",
  pedidos: [
    { total: 120.50, fecha: ISODate("2026-10-01") },
    { total: 80.00,  fecha: ISODate("2026-10-03") }
  ],
  preferencias: { tema: "oscuro", newsletter: true }
})
db.clientes.insertOne({ nombre: "María Huamán", ciudad: "Lima" })
```

▶ Resultado en mongosh

```
{ acknowledged: true,
  insertedId: ObjectId('6702a9c4e1b2f3a4c5d6e7f8') }
```

Embebido

Los pedidos viajan dentro del cliente: se leen en una sola operación, sin JOIN.

Esquema flexible

María no tiene pedidos ni preferencias y el documento sigue siendo válido.

JSON / BSON

El dato llega casi igual a tu front-end en JavaScript.

EJEMPLO 2 · MONGODB

Consultar con un pipeline de agregación

```
db.clientes.aggregate([
  { $match: { ciudad: "Lima" } },
  { $project: {
      _id: 0,
      nombre: 1,
      pedidos: { $size: { $ifNull: ["$pedidos", []] } },
      gastado: { $sum: "$pedidos.total" }
  } }
])
```

▶ Resultado

```
[
  { nombre: "Ana Quispe",
    pedidos: 2, gastado: 200.5 },
  { nombre: "María Huamán",
    pedidos: 0, gastado: 0 }
]
```

$match ≈ WHERE **$project ≈ SELECT** **$sum ≈ SUM** El mismo reporte que en SQL, sin JOIN y ya en formato JSON.

TRANSFORMACIÓN · COMPARATIVA

Relacional frente a NoSQL

Criterio	Relacional (SQL)	NoSQL
Modelo	Tablas con filas y columnas	Documentos, clave-valor, columnas o grafos
Esquema	Rígido, definido antes de insertar	Flexible, puede variar por documento
Relaciones	Claves foráneas y JOIN	Datos embebidos o referencias manuales
Escalado	Principalmente vertical	Horizontal (sharding) por diseño
Consistencia	ACID fuerte	Eventual o configurable (BASE)
Consulta	SQL estándar	API o lenguaje propio de cada motor
Ideal para	Pagos, inventario, ERP, matrículas	Catálogos, chat, IoT, tiempo real

La frontera se difumina: PostgreSQL guarda JSON (JSONB) y MongoDB admite transacciones multi-documento.

TRANSFORMACIÓN · VOLVAMOS AL RETO

Una base de datos para cada necesidad

Dato de la app de delivery	Modelo recomendado	Por qué
Pedidos y pagos	Relacional · PostgreSQL	Transacciones ACID e integridad
Ubicación del repartidor	Clave-valor · Redis	Miles de escrituras por segundo
Chat cliente–repartidor	Documentos · Firestore	Mensajes flexibles en tiempo real
Reseñas e historial	Documentos o JSONB	Estructura variable, mucha lectura

Esto se llama **persistencia políglota**. Regla práctica: empieza relacional y añade NoSQL cuando un requisito concreto lo justifique.

TRANSFORMACIÓN · BACK-END COMO SERVICIO

BaaS: el back-end ya construido, listo para consumir

Un proveedor en la nube ofrece los servicios de servidor más comunes mediante APIs y SDK. Tú **configuras** y los **consumes** desde el front-end.

Base de datos
Con API REST o SDK generada

Autenticación
Email, Google, GitHub, tokens JWT

Archivos
Imágenes y documentos con permisos

Funciones
Lógica propia sin administrar servidor

Tiempo real
Cambios enviados al instante

TRANSFORMACIÓN · BACK-END COMO SERVICIO

¿Qué construyes tú y qué te da el BaaS?

Back-end propio

Tú programas y operas:

- Servidor y API REST
- Conexión y migraciones de la BD
- Registro, login y tokens
- Hosting, escalado y respaldos

Con BaaS

Tú te enfocas en:

- El modelo de datos
- Las reglas de seguridad
- La experiencia en el front-end
- Lógica especial con funciones

Ventajas

Prototipos en horas, escalado gestionado, autenticación lista y plan gratuito para aprender.

Riesgos

Dependencia del proveedor (lock-in), costos al crecer y menos control sobre lógica compleja.

TRANSFORMACIÓN · PLATAFORMAS

Supabase, Firebase y Appwrite

Criterio	Supabase	Firebase	Appwrite
Base de datos	PostgreSQL (relacional)	Firestore (documentos)	API de documentos sobre MariaDB
Código abierto	Sí	No	Sí
Autohospedable	Sí	No	Sí
Seguridad de datos	Row Level Security (SQL)	Security Rules	Permisos por colección y documento
Tiempo real	Sí	Sí	Sí
Plan gratuito	Sí	Sí (Spark)	Sí

En el laboratorio usaremos **Supabase**: es relacional, así que aplicamos todo lo de SQL y además nos da API REST, autenticación y tiempo real.

EJEMPLO 3 · SUPABASE

La tabla ya es una API: consulta desde JavaScript

```
// src/lib/supabase.js
import { createClient } from '@supabase/supabase-js'
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)

// pedidos de 50 soles o más, con el nombre del cliente
const { data, error } = await supabase
  .from('pedidos')
  .select('id, total, clientes ( nombre )')
  .gte('total', 50)
  .order('total', { ascending: false })
```

▶ data

```
[
  { "id": 1, "total": 120.5,
    "clientes":
      { "nombre": "Ana Quispe" } },
  { "id": 2, "total": 80,
    "clientes":
      { "nombre": "Ana Quispe" } }
]
```

Petición REST equivalente que envía el SDK (lo que viste en la Unidad 1):

`GET /rest/v1/pedidos?select=id,total,clientes(nombre)&total=gte.50&order=total.desc`

EJEMPLO 4 · FIREBASE FIRESTORE

NoSQL en tiempo real: el chat del delivery

```
import { collection, addDoc, onSnapshot,
        query, where } from 'firebase/firestore'
import { db } from './firebase'

// escribir un mensaje del chat
await addDoc(collection(db, 'mensajes'), {
  pedidoId: 'P-001', autor: 'repartidor',
  texto: 'Estoy a 5 minutos', creado: Date.now()
})

// escuchar cambios en tiempo real
const q = query(collection(db, 'mensajes'),
              where('pedidoId', '==', 'P-001'))
onSnapshot(q, (snap) =>
  snap.docChanges().forEach((c) =>
    console.log(c.type, c.doc.data().texto)))
```

▶ Consola del navegador

```
added Estoy a 5 minutos
added Ya llegué, estoy
      en la puerta
```

Sin recargar

El segundo mensaje lo escribió otro usuario: onSnapshot lo recibe al instante porque mantiene un canal abierto con el servidor.

TRANSFORMACIÓN · CONSTRUCCIÓN DE UNA BD

Seis pasos para construir una base de datos

1

Requisitos

Qué datos se guardan y qué preguntas debe responder la app.

2

Modelo conceptual

Entidades, atributos y relaciones en un diagrama E-R.

3

Modelo lógico

Tablas, claves primarias y foráneas, cardinalidades.

4

Normalización

1FN, 2FN y 3FN para eliminar redundancia.

5

Modelo físico

Script SQL: tipos, restricciones e índices.

6

Seguridad y prueba

Políticas de acceso, datos semilla y consultas de verificación.

Lo aplicamos de punta a punta con un caso: **TaskFlow**, un gestor de tareas para equipos.

PASOS 1 Y 2 · REQUISITOS

Caso TaskFlow: de los requisitos a las entidades

REQUISITOS DEL CLIENTE

Un **usuario** se registra con su email. Crea **proyectos** y en cada proyecto registra **tareas** con título, estado (pendiente, en progreso o hecha), prioridad del 1 al 3 y fecha límite.

Cada tarea puede tener varias **etiquetas** (ui, backend, urgente...) y una etiqueta se usa en muchas tareas.

Técnica: los **sustantivos** sugieren entidades; los **verbos**, relaciones.

Entidades

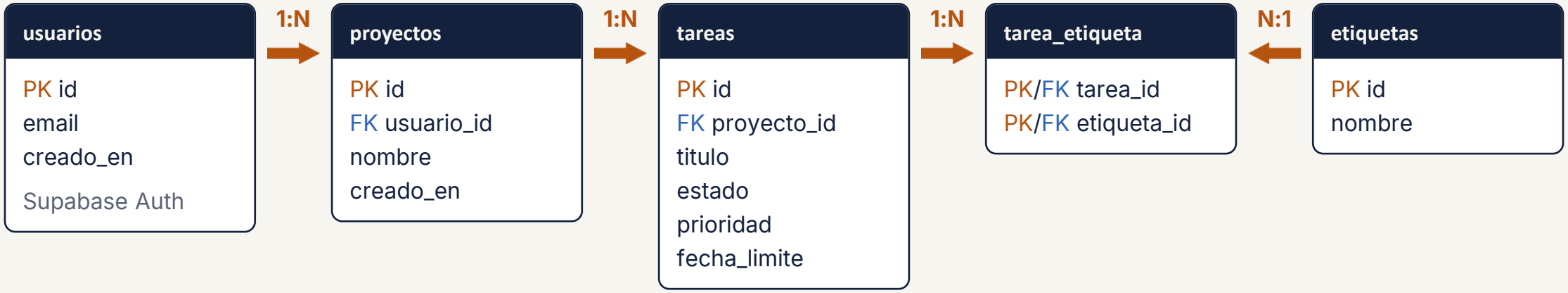
Usuario · Proyecto · Tarea · Etiqueta

Relaciones

Usuario 1:N Proyecto
Proyecto 1:N Tarea
Tarea N:M Etiqueta

PASOS 2 Y 3 · MODELO E-R Y LÓGICO

TaskFlow: cinco tablas y sus relaciones



Leyenda

PK clave primaria · FK clave foránea · 1:N uno a muchos

¿Y la relación N:M?

Tarea ↔ Etiqueta se resuelve con la tabla intermedia tarea_etiqueta, cuya clave primaria combina las dos FK.

PASO 4 · NORMALIZACIÓN

De una planilla con problemas a tablas limpias

Antes: todo en una sola tabla

titulo	proyecto	dueno_email	etiquetas
Diseñar login	App Móvil	ana@mail.pe	ui, urgente
Crear endpoint	App Móvil	ana@mail.pe	backend
Configurar CI	Web Admin	luis@mail.pe	backend, urgente

1FN

Valores atómicos. «ui, urgente» guarda dos datos en una celda → tablas etiquetas y tarea_etiqueta.

2FN

Cada atributo depende de toda la clave. En tarea_etiqueta solo viven las dos FK de su clave compuesta.

3FN

Sin dependencias transitivas. El email depende del usuario, no de la tarea → vive en usuarios.

PASO 5 · MODELO FÍSICO (1/2)

Script SQL en el SQL Editor de Supabase

```sql
create table proyectos (
  id uuid primary key default gen_random_uuid(),
  usuario_id uuid not null default auth.uid()
    references auth.users,
  nombre text not null,
  creado_en timestamptz default now()
);

create table tareas (
  id bigint generated always as identity primary key,
  proyecto_id uuid not null
    references proyectos on delete cascade,
  titulo text not null,
  estado text not null default 'pendiente'
    check (estado in ('pendiente','en_progreso','hecha')),
  prioridad int check (prioridad between 1 and 3),
  fecha_limite date
);
```

gen_random_uuid()

IDs únicos y difíciles de adivinar.

default auth.uid()

El dueño del proyecto se asigna solo al insertar.

on delete cascade

Si borras el proyecto, se borran sus tareas.

check (…)

El estado solo admite tres valores; la prioridad, de 1 a 3.

PASO 5 · MODELO FÍSICO (2/2)

La tabla intermedia y un índice

```
create table etiquetas (
  id serial primary key,
  nombre text unique not null
);

create table tarea_etiqueta (
  tarea_id bigint references tareas on delete cascade,
  etiqueta_id int references etiquetas on delete cascade,
  primary key (tarea_id, etiqueta_id)
);

create index idx_tareas_proyecto on tareas (proyecto_id);
```

▶ SQL Editor

Success. No rows returned

▶ Table Editor

Tabla	Columnas
proyectos	4
tareas	6
etiquetas	2
tarea_etiqueta	2

PASO 6 · DATOS SEMILLA Y PRUEBA

¿La base responde lo que la app necesita?

```
insert into etiquetas (nombre) values ('ui'), ('backend'), ('urgente');
-- (luego: 1 proyecto y 3 tareas creados desde la app, y etiquetados)
select t.titulo, t.estado, string_agg(e.nombre, ', ') as etiquetas
from tareas t
left join tarea_etiqueta te on te.tarea_id = t.id
left join etiquetas e on e.id = te.etiqueta_id
group by t.id, t.titulo, t.estado
order by t.id;
```

▶ **Resultado**

titulo	estado	etiquetas
Diseñar pantalla de login	en_progreso	ui, urgente
Crear endpoint de tareas	pendiente	backend
Configurar despliegue	hecha	NULL

PASO 6 · SEGURIDAD

Row Level Security: cada quien ve solo lo suyo

```
-- 1. Activar seguridad a nivel de fila
alter table proyectos enable row level security;
alter table tareas    enable row level security;

-- 2. Cada usuario gestiona solo sus proyectos
create policy "mis_proyectos" on proyectos
  for all using ( auth.uid() = usuario_id )
  with check ( auth.uid() = usuario_id );

-- 3. ...y solo las tareas de sus proyectos
create policy "mis_tareas" on tareas
  for all using ( exists (
    select 1 from proyectos p
    where p.id = tareas.proyecto_id
      and p.usuario_id = auth.uid() ) );
```

▶ Resultado

La misma consulta **select * from proyectos** ejecutada por:

Sesión	Filas
Ana	2
Luis	1
Sin sesión	0



Clave pública

La anon key viaja en el front-end: cualquiera puede verla. La seguridad real la ponen las políticas RLS.

INTEGRACIÓN CON EL FRONT-END

Hook useTareas: React habla con Supabase

```
import { useEffect, useState } from 'react'
import { supabase } from '../lib/supabase'

export function useTareas(proyectoId) {
  const [tareas, setTareas] = useState([])
  useEffect(() => {
    supabase.from('tareas').select('id, titulo, estado')
      .eq('proyecto_id', proyectoId).order('id')
      .then(({ data }) => setTareas(data ?? []))
  }, [proyectoId])
  const crear = async (titulo) => {
    const { data } = await supabase.from('tareas')
      .insert({ proyecto_id: proyectoId, titulo })
      .select().single()
    setTareas((t) => [...t, data])
  }
  return { tareas, crear }
}
```

▶ Resultado en el navegador (npm run dev)



PRÁCTICA · LABORATORIO GUIADO (25 MIN)

Construye TaskFlow: de la base de datos a la pantalla

#	Actividad	Min
1	Crea un proyecto gratuito en supabase.com y copia la Project URL y la anon key	3
2	Ejecuta en el SQL Editor el script del modelo físico (tablas, índice y etiquetas)	4
3	Activa RLS, crea las dos políticas y un usuario de prueba en Authentication	3
4	npm create vite@latest taskflow -- --template react · npm i @supabase/supabase-js · .env	4
5	Inicia sesión con signInWithPassword y crea tu primer proyecto	3
6	Implementa useTareas: listar y crear tareas	5
7	Cambia el estado de una tarea con update() y verifícalo en el Table Editor	3

Reto (+2 pts): filtra por etiqueta con select('titulo, etiquetas!inner(nombre)').eq('etiquetas.nombre', 'urgente')

PRÁCTICA · ENTREGABLE

Qué entregas y cómo se evalúa

Criterio	Evidencia	Pts
Modelo E-R normalizado a 3FN	Diagrama (imagen o draw.io)	4
Script físico con PK, FK y CHECK	schema.sql + captura del Table Editor	4
Seguridad con RLS	Políticas en schema.sql y prueba con dos usuarios	4
App React: listar, crear y actualizar	Repositorio GitHub + video de 1 minuto	6
Reto: filtro por etiqueta	Funcionalidad visible en la app	2
Total		20

Entrega: [fecha y hora] **Medio: aula virtual** **Modalidad: parejas**

CIERRE · METACOGNICIÓN

Lo que te llevas hoy

1. El modelo se elige por el problema: relacional para integridad; NoSQL para flexibilidad y escala.
2. Un BaaS te da API, autenticación y base de datos listas; la seguridad la defines tú con RLS o reglas.
3. Construir una BD es un proceso: requisitos, E-R, normalización, script físico y seguridad.

RESPONDE EN TU CUADERNO

- ¿Qué paso del modelado te costó más y por qué?
- ¿En qué parte de tu proyecto usarías NoSQL?
- ¿Qué pasaría si olvidas activar RLS?

CIERRE · PARA SEGUIR APRENDIENDO

Próxima sesión y referencias

Próxima sesión

[Tema de la próxima sesión]

Trae TaskFlow funcionando: lo reutilizaremos y ampliaremos.

Referencias

Documentación de Supabase · supabase.com/docs

Documentación de PostgreSQL · postgresql.org/docs

Manual de MongoDB · mongodb.com/docs/manual

Cloud Firestore · firebase.google.com/docs/firestore

Silberschatz, Korth y Sudarshan · Fundamentos de bases de datos, McGraw-Hill